20 preguntas para estudiar Agencia Atenas

Teoría del proyecto y preparación para la defensa

Responde con tus palabras y usa ejemplos de la app. Primero intenta resolver las preguntas sin mirar las respuestas. El solucionario comienza después de la pregunta 20. Puedes ampliar tus respuestas en un cuaderno.

1. El propósito de la app

¿Qué problema busca resolver Agencia Atenas? Explica por qué una oportunidad comercial necesita más información que el nombre y la fecha de un evento.

2. Roles y permisos

¿Por qué existen un ejecutivo y un supervisor? Si ocultamos al ejecutivo el botón para confirmar, ¿eso basta para impedir una confirmación no autorizada? Justifica.

3. La separación del código

¿Por qué no ponemos toda la lógica en la pantalla? Explica con tus palabras qué hacen la pantalla, el ViewModel, el repositorio y el DAO en este proyecto.

4. StateFlow y persistencia

¿Cómo se entera la pantalla de que cambió el nombre escrito o la lista de oportunidades? ¿Por qué tener un dato en StateFlow no equivale a guardarlo en Room?

5. Crear y editar

Describe el recorrido al pulsar Guardar oportunidad. ¿Cómo distingue la app una creación de una edición y por qué editar no debería generar un registro nuevo?

Preguntas 6 a 10

Explica qué ocurre, cómo funciona y por qué se eligió esa solución.

6. Datos relacionados y normalización

¿Por qué Organización y Contacto están en tablas separadas de Oportunidad? Explica para qué sirven los identificadores, las claves foráneas y la relación con el contacto principal.

7. Responsable actual e historial

Si una oportunidad pasa de ejecutivo1 a ejecutivo2, ¿por qué sus gestiones anteriores no deberían cambiar de autor? ¿Qué ocurre con el contacto histórico si cambia el contacto principal?

8. Estado, seguimiento y tarea pendiente

¿En qué se diferencian el estado de una oportunidad, una gestión registrada y su próxima acción? Si una llamada vence hoy, ¿por qué no se completa automáticamente al terminar el día?

9. Búsqueda y filtros

¿Qué significa combinar filtros? Si buscamos una feria de prioridad Alta en una comuna y no aparecen resultados, ¿qué podemos concluir y cómo volvemos a ver el conjunto disponible?

10. Datos obligatorios y desconocidos

¿Por qué algunos campos son obligatorios y otros se pueden dejar vacíos? En los indicadores del resultado de una activación, ¿qué diferencia hay entre dejar un dato vacío y escribir cero?

Preguntas 11 a 15

Explica qué ocurre, cómo funciona y por qué se eligió esa solución.

11. Fechas visibles y fechas guardadas

¿Por qué mostramos las fechas como DD/MM/AAAA pero guardamos AAAA-MM-DD? Explica también por qué no basta con que el texto de una fecha tenga barras y números.

12. La validación del teléfono

¿Cómo valida el teléfono nuestra app? Explica qué ocurre con un valor vacío, con +56 9 1234 5678 y con un texto que incluya letras. ¿Qué limitación tiene esta validación?

13. Errores y conservación del formulario

¿Por qué las validaciones se mantienen fuera de los Composables y los errores aparecen debajo de cada campo? ¿Qué debería pasar con los datos escritos cuando una validación o un guardado falla?

14. Apertura de la cámara

¿Cómo toma una foto la app usando TakePicture y FileProvider? ¿Por qué comparte una URI content:// en lugar de una ruta privada y no solicita el permiso CAMERA para este recorrido?

15. Archivos y cancelación de una foto

¿Dónde se conservan las fotos y cómo se relacionan con una oportunidad? Explica qué pasa al cancelar la cámara y por qué no bastaría con guardar la imagen solo en memoria.

Preguntas 16 a 20

Explica qué ocurre, cómo funciona y por qué se eligió esa solución.

16. Integración con el calendario

¿Qué hace Añadir evento al calendario? ¿Abrir el editor significa que el evento ya quedó guardado? Explica qué ocurre si no se indicó horario o si exportamos varias veces el mismo evento.

17. Animaciones con una función

¿Cuáles son las tres interacciones animadas de las fotos y qué mecanismos utilizan? Explica por qué una animación no debe decidir si una fotografía existe o está guardada.

18. Cerrar sesión y compartir el proyecto

¿Qué diferencia hay entre cerrar una oportunidad y cerrar sesión? Si enviamos el ZIP del proyecto al compañero, ¿por qué no aparecen automáticamente nuestras oportunidades en su emulador?

19. Cambios de estructura en Room

¿Por qué una versión nueva del código puede fallar al abrir una base de datos antigua? Explica qué decisión tomó el equipo sobre las migraciones y cómo probar la versión actual sin perder la instalación anterior.

20. Pruebas y revisión antes de entregar

¿Qué diferencias hay entre una prueba unitaria, una instrumental, una revisión con Lint y una prueba manual? Da ejemplos de este proyecto y explica por qué una compilación exitosa no basta.

Solucionario

Respuestas 1 a 4

Son respuestas orientativas basadas en el proyecto actual. No necesitas repetirlas de memoria: lo importante es explicar la idea y justificarla con un ejemplo.

1. El propósito de la app

La app organiza el trabajo comercial asociado a posibles participaciones en eventos. El nombre y la fecha indican qué actividad existe, pero no explican quién la atiende, con quién se ha hablado, qué respuesta hubo ni qué falta hacer. Por eso una oportunidad reúne antecedentes del evento, organización, contacto, responsable, prioridad, estado y gestiones. Permite retomar el trabajo y registrar su resultado. Crear una oportunidad no significa que la participación ya esté confirmada.

2. Roles y permisos

El ejecutivo gestiona sus oportunidades asignadas. El supervisor consulta el conjunto, asigna responsables y puede confirmar o descartar una participación. La separación refleja responsabilidades distintas. Ocultar un botón ayuda a mostrar las opciones correctas, pero no garantiza el permiso de una operación. El repositorio también comprueba el usuario vigente, el acceso a la oportunidad y la transición solicitada antes de escribir. Así la regla no depende solamente de que la pantalla muestre u oculte un control.

3. La separación del código

La pantalla de Compose presenta los datos y recibe las acciones del usuario. El ViewModel mantiene el estado y coordina operaciones como validar y guardar. El repositorio reúne el acceso a los datos y aplica reglas de autorización. El DAO contiene las operaciones de consulta y escritura de Room. Separarlos permite seguir cada responsabilidad, reutilizar reglas y probar partes sin depender de toda la interfaz. Por ejemplo, cambiar el diseño de un formulario no debería exigir reescribir sus consultas de base de datos.

4. StateFlow y persistencia

El ViewModel publica el estado mediante StateFlow y la pantalla lo observa con collectAsStateWithLifecycle. Al recibir otro estado, Compose actualiza la parte de la interfaz que depende de él. Las consultas observables de Room también pueden alimentar ese estado cuando cambian los registros. StateFlow mantiene información observable en memoria, por lo que no sustituye la persistencia. Para conservar una oportunidad después de cerrar la app hay que guardarla en Room. Observar un valor y escribirlo en la base son operaciones distintas.

Solucionario - continuación

Respuestas 5 a 8

5. Crear y editar

La pantalla envía la acción al ViewModel. Este toma los datos del formulario, los valida y solicita el guardado al repositorio. El repositorio comprueba permisos y relaciones, y utiliza el DAO. Un id igual a cero identifica una creación y se inserta una fila con un nuevo identificador. Un id existente permite actualizar esa misma oportunidad, después de comprobar el acceso. Tras confirmar el guardado, se abre el detalle. Si falla, se muestra el error y se conserva el formulario. Editar mantiene el identificador para no duplicar la oportunidad.

6. Datos relacionados y normalización

Separar las entidades evita copiar la organización y todos sus contactos cada vez que se crea una oportunidad. La clave primaria identifica un registro aunque su nombre se repita. Las claves foráneas enlazan registros existentes y ayudan a impedir relaciones inválidas. El contacto pertenece a una organización y la oportunidad puede seleccionar uno de sus contactos como principal mediante OportunidadContacto. Así se reutilizan los datos y se consulta su información relacionada. La organización puede quedar sin seleccionar cuando todavía se desconoce.

7. Responsable actual e historial

El responsable actual indica quién debe continuar el trabajo. El autor de una gestión identifica quién la realizó y forma parte del historial. Reasignar la oportunidad cambia el acceso y la responsabilidad actuales, pero no debe atribuir el trabajo previo a otra persona. De manera similar, cada seguimiento conserva el contacto con quien se registró esa gestión. Cambiar después el contacto principal no mueve las conversaciones anteriores al nuevo contacto. El historial mantiene el contexto de lo realizado, dentro de los permisos vigentes de consulta.

8. Estado, seguimiento y tarea pendiente

El estado resume la situación comercial, por ejemplo En espera de respuesta. Una gestión documenta un hecho, como una llamada y su resultado. La próxima acción es un trabajo programado con fecha. Llegar a esa fecha no demuestra que se haya realizado: una tarea sin resolución queda para hoy o pasa a atrasada. Registrar resultado crea otra gestión y la vincula como resolución de la pendiente, conservando ambas. Cambiar el estado de la oportunidad tampoco sustituye ese registro de resultado.

Solucionario - continuación

Respuestas 9 a 12

9. Búsqueda y filtros

Combinar filtros significa exigir que una oportunidad cumpla todos los criterios activos al mismo tiempo, además de pertenecer al conjunto autorizado de la cuenta. Si no hay resultados, no hay coincidencias con esa combinación. No significa que se hayan borrado las oportunidades. Podemos revisar los criterios o usar Limpiar filtros, que también limpia la búsqueda por nombre. Ocultar el bloque de filtros no los desactiva. El supervisor dispone además del filtro por responsable, mientras el ejecutivo sigue limitado a sus oportunidades.

10. Datos obligatorios y desconocidos

Los obligatorios aportan la información mínima necesaria para trabajar con el registro. Los opcionales permiten incorporar detalles cuando se conocen, sin inventarlos para completar el formulario. En los indicadores numéricos, vacío representa un dato desconocido y se conserva como null. Cero significa que conocemos el resultado y la cantidad fue cero. No son equivalentes. Si se completa un campo opcional, igualmente debe respetar su formato y sus reglas, por ejemplo que un número de asistentes sea entero y no negativo.

11. Fechas visibles y fechas guardadas

DD/MM/AAAA facilita la lectura y el ingreso para los usuarios del proyecto. AAAA-MM-DD, con sus componentes completos, permite que el orden del texto coincida con el orden cronológico al comparar o consultar fechas. Una forma aparente no garantiza una fecha real: 31/02/2026 tiene barras y números, pero no existe. La conversión comprueba el formato y utiliza un análisis estricto, sin ajustar fechas imposibles. Además, las reglas del formulario revisan relaciones como que el término no sea anterior al inicio.

12. La validación del teléfono

El teléfono es opcional, por lo que vacío se acepta. Si hay texto, solo se permiten dígitos del 0 al 9, espacios, paréntesis, guiones y el signo +. Se cuentan únicamente los dígitos y deben ser entre 8 y 15. Por eso +56 9 1234 5678 cumple y un texto con letras falla. La función revisa un formato general: no comprueba que el número exista ni que sea chileno. Actualmente tampoco restringe el signo + a una sola aparición al principio.

Solucionario - continuación

Respuestas 13 a 16

13. Errores y conservación del formulario

Las funciones de validación separadas permiten reutilizar y probar las reglas sin depender del dibujo de la pantalla. El ViewModel recoge los errores y los publica en el estado. El campo utiliza isError, supportingText y los indicadores visuales para explicar qué corregir. Si la validación o el guardado falla, la app conserva lo escrito y muestra un mensaje. No debe navegar como si hubiese guardado ni limpiar el borrador. Así el usuario puede corregir o reintentar sin tener que reconstruir toda la información.

14. Apertura de la cámara

Primero se prepara un archivo de destino. FileProvider permite representarlo mediante una URI content:// y compartirlo con acceso temporal. TakePicture abre una aplicación de cámara, que escribe la captura en ese destino y devuelve si la operación tuvo éxito. No se expone toda la carpeta privada ni se comparte una ruta file:// directamente. En este recorrido Agencia Atenas delega la captura a la aplicación de cámara, en lugar de acceder directamente al hardware, por lo que no declara CAMERA. Debe existir una aplicación capaz de atender la captura.

15. Archivos y cancelación de una foto

Las fotos se conservan como archivos en el almacenamiento privado persistente, dentro de evidencias y una carpeta identificada por el id de la oportunidad. No se guardan como bitmaps dentro de SQLite. Antes de capturar se crea un archivo pendiente con un nombre único. Si se cancela, se elimina ese archivo sin agregar una foto al listado. Si la captura es válida, se conserva como JPG. Guardar solo en memoria perdería la imagen al terminar el proceso. SavedStateHandle ayuda a recuperar la ruta pendiente si Android recrea el proceso durante la captura.

16. Integración con el calendario

La app envía un Intent ACTION_INSERT con título, descripción, ubicación y fechas al calendario del dispositivo. La persona revisa y decide guardar o cancelar allí. Abrir el editor no confirma el guardado. Si no se indicó horario, se prepara un evento de día completo. La aplicación externa puede necesitar una cuenta configurada y guardar varias veces la exportación puede producir duplicados. Agencia Atenas no escribe directamente en el calendario, por lo que este recorrido no solicita READ_CALENDAR ni WRITE_CALENDAR. Los avisos de próximas gestiones siguen siendo internos.

Solucionario - continuación

Respuestas 17 a 20

17. Animaciones con una función

La vista previa aparece mediante AnimatedVisibility con desvanecimiento y despliegue. La ampliación se abre y cierra con AnimatedVisibility y un cambio de opacidad, manteniendo fijo el tamaño del diálogo. El contador cambia mediante Crossfade. Son tres interacciones realizadas con dos mecanismos principales. Dan respuesta visual a un cambio real o a una acción del usuario, pero no guardan archivos ni crean evidencia. El repositorio y el estado determinan qué fotos existen. Cancelar la cámara no debe aumentar el contador ni mostrar una captura inexistente.

18. Cerrar sesión y compartir el proyecto

Cerrar una oportunidad termina su recorrido comercial según las reglas de estado y conserva su historial. Cerrar sesión termina el acceso de la cuenta y vuelve al ingreso, sin borrar los registros ni las fotos. Cada instalación conserva sus propios archivos y su base local. El ZIP del código no contiene automáticamente los datos privados del emulador, y este proyecto no tiene sincronización entre dispositivos. Por eso el compañero puede ejecutar el mismo código y disponer de otros registros. El acceso al volver a ingresar depende del rol y la asignación actuales.

19. Cambios de estructura en Room

El código y la base instalada deben coincidir en su estructura. Agregar o cambiar tablas y columnas no transforma automáticamente los datos antiguos. El proyecto utiliza el esquema 5 y el equipo eligió una instalación limpia sin migraciones. No incluye una conversión de bases anteriores ni un borrado automático para reemplazarlas. Para conservar la instalación vieja, se puede probar en otro emulador. Borrar el almacenamiento también permite empezar de nuevo, pero elimina sus registros y fotografías. Es una limitación de esta entrega que debe explicarse al ejecutarla.

20. Pruebas y revisión antes de entregar

Una prueba unitaria revisa lógica aislada, como formatos de fecha o reglas de estados, normalmente en la computadora. Una instrumental se ejecuta en Android y puede comprobar consultas y relaciones reales de Room. Lint analiza el proyecto para detectar posibles problemas sin recorrer la app como usuario. Una prueba manual verifica el recorrido y la presentación, por ejemplo tomar una foto, revisar permisos o guardar un evento en el calendario. Compilar confirma que se puede construir la app, pero no demuestra que los permisos, la persistencia o todas las interacciones se comporten correctamente.